

Advanced Topics in Computer Graphics I

Markov Chain Monte Carlo Light Transport Simulation



Prof. Dr. Reinhard Klein

Institute of Computer Science II - Computer Graphics
University of Bonn

June 3, 2025

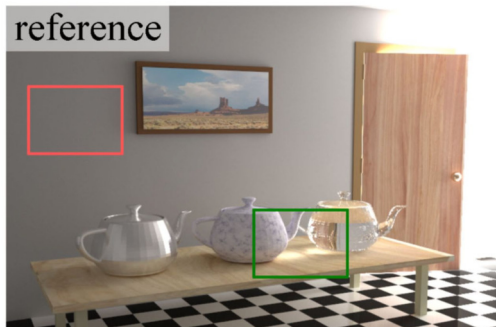


- ▶ Ordinary Monte Carlo (OMC)
- ▶ Photon Mapping
- ▶ Markov Chain Monte Carlo (MCMC)



Markov Chain Monte Carlo Methods

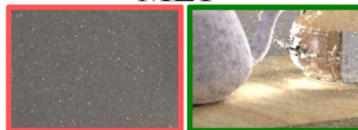




BDPT



MLT



Equal-time comparison (12 min.) of Metropolis Light Transport¹ (MLT) and bidirectional path tracing (BDPT). In this case, MLT produces much less noise thanks to its ability to generate more paths that pass through the door ajar and can thus contribute to the image. Taken from².

¹E. Veach and L. J. Guibas. "Metropolis light transport". In: *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*. 1997, pp. 65–76.

²M. Šik and J. Křivánek. "Survey of Markov chain Monte Carlo methods in light transport simulation". In: *IEEE transactions on visualization and computer graphics* 26.4 (2018), pp. 1821–1840.



Overview:

- ▶ World Space Methods Metropolis Light Transport³
- ▶ Primary sample space⁴
- ▶ Multiplexed Metropolis Light Transport⁵
- ▶ Population Monte Carlo^{6,7}

³E. Veach and L. J. Guibas. "Metropolis light transport". In: *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*. 1997, pp. 65–76.

⁴C. Kelemen et al. "A simple and robust mutation strategy for the metropolis light transport algorithm". In: *Computer Graphics Forum*. Vol. 21. 3. Wiley Online Library. 2002, pp. 531–540.

⁵T. Hachisuka et al. "Multiplexed metropolis light transport". In: *ACM Transactions on Graphics (TOG)* 33.4 (2014), pp. 1–10.

⁶D. Cline et al. "Energy redistribution path tracing". In: *ACM Transactions on Graphics (TOG)* 24.3 (2005), pp. 1186–1195.

⁷Y.-C. Lai et al. "Photorealistic image rendering with population monte carlo energy redistribution". In: *Proceedings of the 18th Eurographics conference on Rendering Techniques*. 2007, pp. 287–295.



Metropolis Light Transport





Metropolis Light Transport (MLT) is an extension of the path tracing algorithm that leverages the Metropolis-Hastings algorithm, a Markov Chain Monte Carlo (MCMC) technique.

It starts from the measurement equation

$$\begin{aligned} I_j &= \int_{\mathcal{P}} \underbrace{W_e^{(j)}(x_0 \rightarrow x_1) G(x_0 \leftrightarrow x_1) T(\bar{x}) L_e(x_{k+1} \rightarrow x_k)}_{f_j(\bar{x})} d\mu(\bar{x}) \\ &= \int_{\mathcal{P}} f_j(\bar{x}) d\mu(\bar{x}) = \sum_{k=1}^{\infty} \int_{\mathcal{P}^k} f_j(\bar{x}) d\mu(\bar{x}), \end{aligned}$$

where I_j is the intensity of the j th pixel, f_j is the **measurement contribution function** and

$$\mathcal{P} = \bigcup_{k \geq 1} \mathcal{P}^k$$

is the space of all possible light transport paths of all length k that connect light sources to the camera through the scene.



Metropolis Light Transport (MLT) is based on the observation that rendering involves solving many related integrals simultaneously.

For this purpose we decompose the **measurement contribution function**

$$f_j(\bar{x}) = h_j(\bar{x})f(\bar{x}),$$

where h_j represents the filter function for pixel j , and f represents all the other factors of f_j , which are the same for all pixels.

$$I_j = \int_{\mathcal{P}} f_j(\bar{x})d\mu(\bar{x}) = \int_{\mathcal{P}} h_j(\bar{x})f(\bar{x})d\mu(\bar{x}),$$

where only the reconstruction filter h_j changes between pixels j .

Note that h_j depends only on the first edge x_0, x_1 of the path $\bar{x}$, which was called the **lens edge** by Veach.



An image is computed by sampling N paths $\bar{x}_i$ according to some density function p

$$I_j \approx \frac{1}{N} \sum_{i=1}^N h_j(\bar{x}_i) \frac{f(\bar{x}_i)}{p(\bar{x}_i)} = \frac{1}{N} \sum_{i=1}^N h_j(\bar{x}_i) C_j(\bar{x}_i), \quad (1)$$

where $C_j(\bar{x}) = f(\bar{x})/p(\bar{x})$ is the **path contribution**.

Notice that if we could take samples according to the density function $p = \frac{1}{b} f$ (where $b = \int f(\bar{x}) d\bar{x}$ is the normalization constant, the estimate for each pixel would be

$$I_j \approx \frac{1}{N} \sum_{i=1}^N b h_j(\bar{x}_i). \quad (2)$$

This equation can be evaluated efficiently for all pixels at once, since each path contributes to only a few pixel values.

If the function h_j does not introduce too much variance, this estimator should theoretically have low variance.



Metropolis light transport as introduced by Veach and Guibas Veach and Guibas, 1997 use the normalized scalar luminosity of the path throughput $f(\bar{x})$

$$p(\bar{x}) = \frac{f^*(\bar{x})}{\int f^*(\bar{x})d\bar{x}}$$

as the target distribution in the Metropolis Hastings Algorithm, to generate a sequence of samples $\bar{x}_i$.

Based on this sequence the pixel values are calculated by

$$I_j \approx \frac{1}{N} \sum_{i=1}^N h_j(\bar{x}_i) \frac{f(\bar{x}_i)}{f^*(\bar{x}_i)} \underbrace{\left(\int f^*(\bar{x})d\bar{x} \right)}_b = \frac{b}{N} \sum_{i=1}^N h_j(\bar{x}_i) \frac{f(\bar{x}_i)}{f^*(\bar{x}_i)} \quad (3)$$

Unlike in path tracing, path samples $\bar{x}_i$ are shared among pixels in MLT: samples are projected to the image plane and contribute to pixels with non-zero $h_j(\bar{x}_i)$.

Therefore, they can be used for estimating the integrals of all pixels at the same time.

The common scaling factor

$$b = \int f^*(\bar{x}) d\bar{x}$$

(related to the brightness of the image) is estimated by separate MC integration such as Path Tracing or Bidirectional Path Tracing.

The MLT updates the current path $\bar{x}_i$ using MCMC

$$a(\bar{x}_j|\bar{x}_i) = \min\left(1, \frac{f^*(\bar{x}_j)t(\bar{x}_i|\bar{x}_j)}{f^*(\bar{x}_i)t(\bar{x}_j|\bar{x}_i)}\right)$$



Algorithm: Metropolis Light Transport

Generate path seeds

Approximate $b = \int f^*(z) dz$ from the seeds

Find z_1 from the seeds using $f^*(z)$

for $i = 1$ to M **do**

 Based on z_i , sample a tentative point z_t using $t(z_t|z_i)$

$$a(z_t|z_i) = \min \left\{ \frac{f^*(z_t)T(z_t|z_i)}{f^*(z_i)t(z_t|z_i)}, 1 \right\}$$

 Select pixel j to which z_i contributes

$$\Phi_j + = \frac{b}{M} I(z_i) f^*(z_i) (1 - a(z_t|z_i))$$

 Select pixel k to which z_t contributes

$$\Phi_k + = \frac{b}{M} I(z_t) f^*(z_t) a(z_t|z_i)$$

 Generate random number r in $[0, 1]$.

if $r < a(z_t|z_i)$ **then**

$$z_{i+1} = z_t$$

else

$$z_{i+1} = z_i$$

end if

end for

Veach and Guibas suggested three different **local mutations** and one **global mutation**.

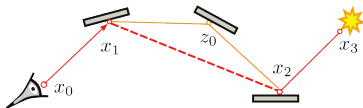
1. **Lens perturbation**, can be effectively used for perturbing paths containing specular surfaces directly visible from the camera
2. **Caustic perturbation**, suited for caustics
3. **Multichain perturbation**, which perform lens and caustic perturbations at once in order to handle difficult to sample paths (such as reflected caustics)
4. Ergodicity is ensured by a global mutation called **bidirectional mutation**, which effectively replaces any subpath of the current path.



The **lens perturbation** (left) creates a new path by changing the direction from a selected vertex x_0 towards the next vertex closer to a light source x'_1 , the vertex x_2 is then connected to x'_1 . The **caustic perturbation** (right) changes the direction towards the camera from a vertex closer to a light source x_2 and connecting x_0 to a new vertex x'_1 .

Bidirectional mutations are the foundation of the MLT algorithm. The **basic idea** is to choose a subpath of the current path $\bar{x}$, and replace it with a different subpath.

The random mutation step deletes the edge x_1x_2 from the path $\bar{x} = x_0x_1x_2x_3$ and replaces it by a new vertex z_0 by casting a ray from x_1 , so that the new path is $\bar{y} = x_0x_1z_0x_2x_3$.



- Choose subpath to delete: Given $\bar{x} = x_0 \dots x_k$ assign a deletion probability $p_d[l; m] = p_{d_1}(l - m) \cdot p_{d_2}(x_l \dots x_m)$ to each subpath $x_l \dots x_m$, $-1 \leq l < m \leq k + 1$ and sample l, m using this probability.
- Generate connecting subpath: Assign a probability to the number of vertices l' and m' to be added to each side of the two remaining paths. $p_a[l'; m'] = p_{a_1}(k_a = l' + m' + 1) \cdot p_{a_2}(l')$. The new vertices are added one at a time to the appropriate subpath by first sampling a direction according to the BSDF at the current subpath endpoint, followed by casting a ray to find the first surface intersected.
- Join the subpaths together: check the visibility between their endpoints. If obstructed, the mutation is immediately rejected.



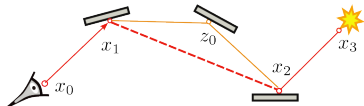
Parameter values In their examples Veach and Guibas had chosen the following probabilities:

- ▶ For $p_{d;1}[k_d]$ of deleting a subpath of length $k_d = m - l$, they used $p_{d;1}[1] = 0.25$, $p_{d;1}[2] = 0.5$, and $p_{d;1}[k_d] = 2^{-k_d}$ for $k_d \geq 3$.
- ▶ For the probability $p_{a;1}[k_a]$ of adding a subpath of length k_a , they used $p_{a;1}[k_d] = 0.5$, $p_{a;1}[k_d - 1] = 0.15$, and $p_{a;1}[k_d - j] = 0.2(2^j)$ for $j \geq 2$.
- ▶ For the acceptance probabilities they observed that

$$a(\bar{x}_j|\bar{x}_i) = \frac{R(\bar{x}_j)}{R(\bar{x}_i)}, \text{ where } R(\bar{x}_j) = \frac{f(x_j)}{t(\bar{x}_j|\bar{x}_i)}$$

and $t(\bar{x}_j|\bar{x}_i)$ is the product of the discrete probability $p_d[l, m]$ for deleting the subpath $x_l, \dots, x_m$ and the probability density for generating the $l' + m'$ new vertices of $\bar{x}_j$.

Example: The random mutation step deletes the edge from the path. Let $\bar{x} = x_0x_1x_2x_3$ be a path and suppose that the random mutation step has deleted edge x_1x_2 and replaced it by the new vertex z_0 , so that the new path is $\bar{y} = x_0x_1z_0x_2x_3$. This corresponds to the random choices $l = 1, m = 2, l' = 1, m' = 0$.



Let $p_\sigma(x'|x)$ denote the probability density of sampling with respect to the solid angle and $p_{\sigma^\perp} = \frac{p_\sigma}{|\cos(\theta_0)|}$, where θ_0 is the angle between xx' and the surface normal. Then the probability density for sampling x' is given by $p_{\sigma^\perp}(x'|x)G(x', x)$. Therefore, we get

$$R(\bar{y}|\bar{x}) = \frac{f(\bar{y})}{t(\bar{y}|\bar{x})} = \frac{\rho(x_1)G(x_1, z_0)\rho(z_0)G(z_0, x_2)\rho(x_2)}{p_a[0, 1]p_{\sigma^\perp}(z_0|x_1)G(x_1, z_0) + p_a[0, 1]p_{\sigma^\perp}(x_2|z_0)G(x_2, z_0)}$$

where in the numerator the factors shared between $R(\bar{y}|\bar{x})$ and $R(\bar{x}|\bar{y})$ are omitted. Similar, we find

$$R(\bar{x}|\bar{y}) = \frac{f(\bar{x})}{t(\bar{x}|\bar{y})} = \frac{\rho(x_1)G(x_1, x_2)\rho(x_2)}{p_d[1, 3]p_a[0, 0]}$$



Summary 1

- ▶ For pixel j , integrand is the product of the measurement function h_j of this pixel and the radiance $f(\bar{x})$ carried by this path.
- ▶ In order to use a single process for all pixels, function f^* can be defined as the scalar contribution to any pixel of the screen, that is, f^* mimics only the radiance and will be independent of the pixel measuring function.
- ▶ In this way, the number of samples contributing to a single pixel will be proportional to the luminance of this pixel.
- ▶ Metropolis sampling converges to the desired probability distribution, but at the beginning of the process the samples are not selected with the required probability, which introduces some error in the estimation. This error is called as the start-up bias.
- ▶ Veach recognized that it is worth using also the rejected samples since they also provide illumination information. Note that a tentative sample is accepted with probability a , while the original sample is kept with probability $1 - a$. Replacing this random variable by its mean, both locations can be contributed but the contributions of the tentative sample and the old sample should be weighted with a and $1 - a$, respectively.



Summary 2:

- ▶ The consideration of the complete path throughput is a key advantage over ordinary MC methods, where it would be difficult to define such a PDF. Due to the acceptance probability

$$a(\bar{x}_j|\bar{x}_i) = \min\left(1, \frac{f^*(\bar{x}_j)t(\bar{x}_i|\bar{x}_j)}{f^*(\bar{x}_i)t(\bar{x}_j|\bar{x}_i)}\right)$$

for path updates using MCMC well-designed mutation strategies in MLT locally explore the sample space of $f^*(\bar{x})$ where they are large, MLT is effective at avoiding sampling paths where $f(\bar{x})$ is zero (thus $f^*(\bar{x})$ is zero) due to visibility.

- ▶ Unfortunately, the implementation of original MLT, however, has been commonly considered very challenging due to the intricate MCMC processes such as the mutations and computation of the transition probabilities directly on a path $\bar{x}$. E.g. randomly perturbing all the vertex locations of a given path almost always results in an invalid path, since perturbed vertices will likely be off the scene surfaces.
- ▶ Each mutation strategy therefore has to be carefully designed such that it generates valid paths, while making a reasonable change to the current path.



Summary 3:

- ▶ The Metropolis algorithm generates correlated samples.
- ▶ In case of correlated samples, the standard deviation of the quadrature can be bounded according to the Bernstein theorem

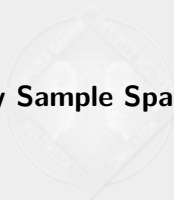
$$\sigma \leq \sigma_{primary} \cdot \sqrt{\frac{1 + 2 \sum_{k=1}^M R(K)}{M}},$$

where, $R(k)$ is an upper-bound of the correlation between $f(z_i)/p(z_i)$ and $f(z_{i+k})/p(z_{i+k})$ and $\sigma_{primary}$ is the standard deviation of $f(z)/p(z)$.

- ▶ If the mutations are small, the correlation between samples will be high. Suppose that the process has found a peak of the integrand. Having made a large perturbation, the scalar contribution function of the tentative sample will be much lower, thus the chance of accepting it will also be low. The point on the peak remains to be the sample point for many steps.
- ▶ Therefore, a single, constant mutation strategy cannot provide an effective algorithm. Instead we need a mutation strategy that is generally large but gets smaller around the peaks of the integrand.
- ▶ The original Metropolis light transport algorithm proposes the random combination of several strategies, each of them being tailored for a particular type of light transfer.

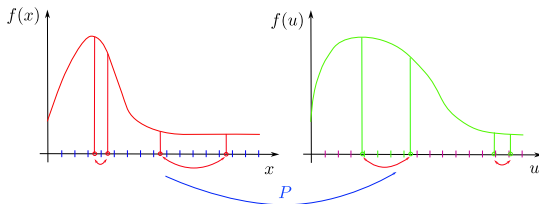


Primary Sample Space MLT





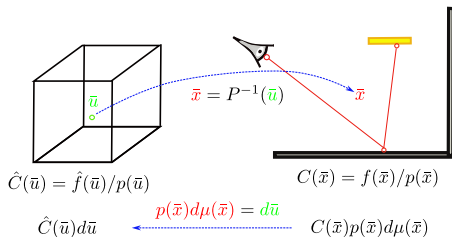
Question: Is it possible to adapt the mutation strategy itself such that the perturbation gets automatically smaller when we are around the peak and thus anticipate that larger perturbations will be rejected?



Remarks

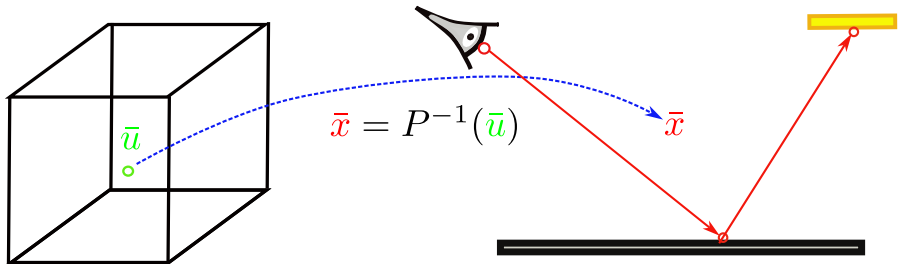
- ▶ If the domain and consequently the integrand is transformed in a way that the local integrand has lower variation, the number of rejections can greatly be reduced (note that the rejection probability is proportional to the ratio of contributions)
- ▶ Such transformation would obviously expand the domain where the original integrand is large and shrink it where the original integrand is small.
- ▶ The basic idea of **PSSMLT** is that such transformation is for free if the random paths are built using importance sampling, e.g. **BRDF sampling**, **light source sampling**, **Russian roulette** and **multiple importance sampling**.

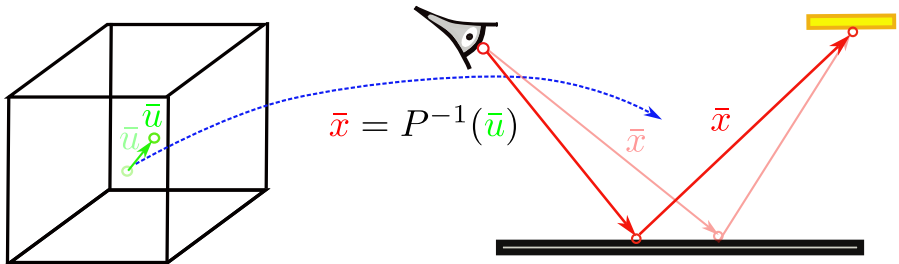
$\bar{u} \in [0, 1]^k$ is a vector of k random numbers mapped to a path $\bar{x}$ of length $k - 1$ according to the inverse cumulative distribution function $P^{-1}(\bar{u})$. The path contribution in each space is $C(\bar{x})$ and $\hat{C}(\bar{u})$. The differential contribution (used for integration) $C(\bar{x})p(\bar{x})d\mu(\bar{x})$ is mapped into $\hat{C}(\bar{u})d\bar{u}$ in primary space.

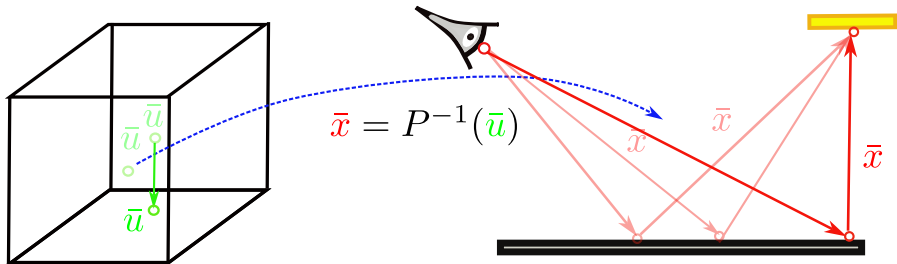


Primary sample space MLT (PSSMLT) significantly simplifies the MCMC process compared to the original MLT method.

⁸C. Kelemen et al. "A simple and robust mutation strategy for the metropolis light transport algorithm". In: *Computer Graphics Forum*. Vol. 21. 3. Wiley Online Library. 2002, pp. 531–540.

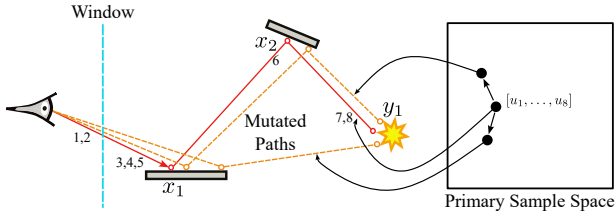








Example: Path tracing with direct light source computation at the end of the path.



- ▶ Using two pseudo-random numbers $u_1; u_2$ a random point on the window is found and a ray is traced through this point to find x_1 .
- ▶ At x_1 we take u_3 to randomly select a BRDF from the elementary BRDFs composing the reflection function at x_1 (e.g. diffuse + specular) and to decide whether or not the walk has to be terminated according to Russian roulette.
- ▶ If we decide to continue the walk, two random numbers $u_4; u_5$ are used to BRDF-sample the direction of a ray starting at x_1 that delivers x_2 .
- ▶ Here we decide again on the termination using u_6 . Suppose the walk is terminated.
- ▶ Direct light source sampling is used to sample y_1 on the surface of the light source using random numbers $u_7; u_8$. If the light source point is visible from x_2 , we have established the following light path: $\bar{x} = (y_1; x_2; x_1; \text{eye})$.



Now assume that in the previous example the elements of the primary sample $\mathbf{u} = (u_1, \dots, u_8)$ are slightly perturbed.

- ▶ if and the decision parameters u_3 and u_6 still fit in the range allowed by the albedos and the perturbation of positions and directions do not alter the respective visibilities, then the new values lead to the same decisions, thus the structure of the path is not altered.
- ▶ However, when one of the new values steps over the albedo limits or alter the visibility relationships, then the remaining part of the sub-path is cut or the previously terminated walk has to be continued.

To ensure the ergodicity condition in the algorithm, completely independent steps are included. These global mutations are called **large steps**.

If a large step is accepted, then Metropolis process starts from a new random seed, which significantly reduces the start-up bias error.

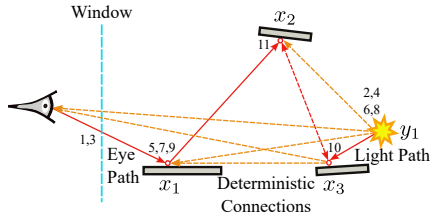
Assuming that $f \star (u_j)$ is quite flat due to BRDF and light source sampling we can use the rather crude approximation

$$a(\bar{x}_j|\bar{x}_i) = \min\left(1, \frac{f^*(\bar{x}_j)t(\bar{x}_i|\bar{x}_j)}{f^*(\bar{x}_i)t(\bar{x}_j|\bar{x}_i)}\right) \approx \min\left(1, \frac{t(\bar{x}_i|\bar{x}_j)}{t(\bar{x}_j|\bar{x}_i)}\right)$$

Thus the acceptance probability will be close to 1 if the mutation function of the large steps are realized with a uniform probability.



Example: The same method can be combined with bidirectional path tracing. This method produces an eye path and a light path and connects all points of the two paths with each other with deterministic shadow rays. Odd and even indices are used to define the eye path and the light path, respectively.



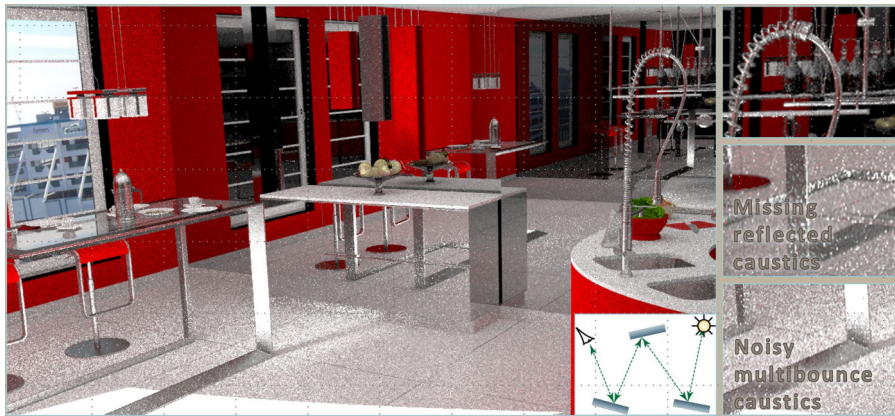
- ▶ This algorithm produces many paths and each of them can also be generated differently by the same algorithm.
- ▶ E.g. the path (eye, x_1, y_1) contains either a random ray between eye, x_1 and a deterministic ray between x_1, y_1 or by a random generation of the ray between y_1, x_1 and a deterministic ray between x_1, eye , both with different probabilities.
- ▶ In order to get the best from these sampling techniques the maximum heuristic of multiple importance sampling is used.



Bidirectional Path Tracing

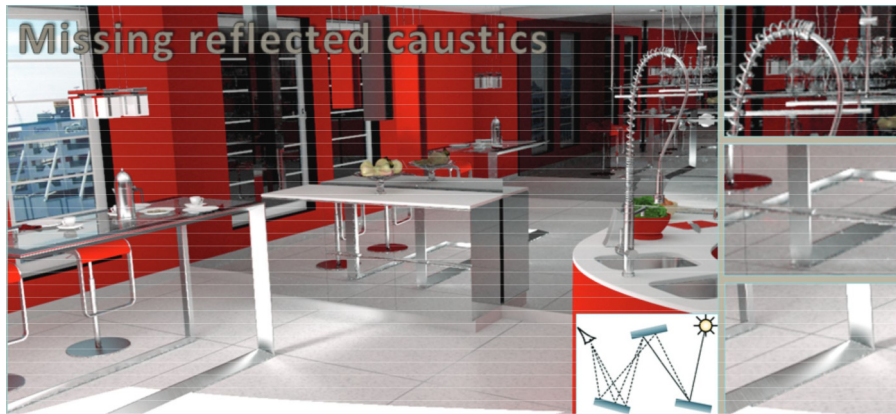


Result Bidirectional Path Tracing, example from Krivanek et al., 2014:





Result, example from Krivanek et al., 2014:



The first MCMC method (also chronologically) is the original Metropolis light transport algorithm Veach and Guibas, 1997 with the original set of mutation strategies.

Result, example from Krivanek et al., 2014:



The first MCMC method (also chronologically) is the original Metropolis light transport algorithm Kelemen et al., 2002 with the original set of mutation strategies.

Result, example from Krivanek et al., 2014:

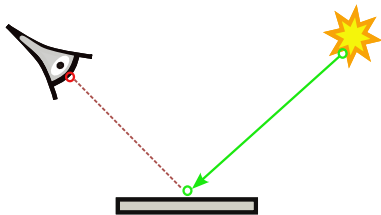
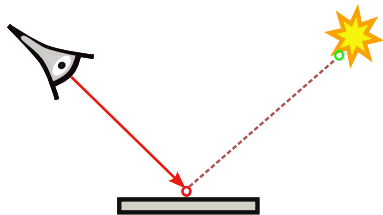
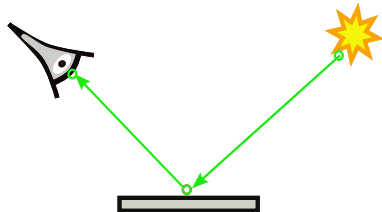
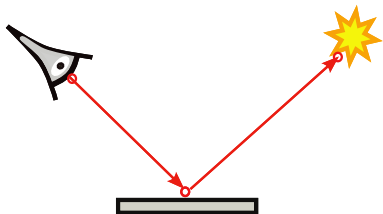


Note that some chains get stuck in complex caustics paths. This happens because of many rejections since the whole path is usually perturbed, thus constantly jumping from its valid form. Also the small step size is specified globally for random numbers and is not adapted for long or highly-occluded paths.

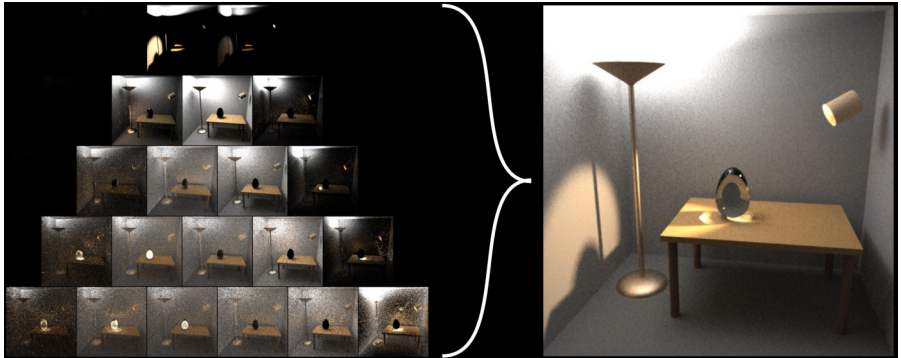


Multiplexed MLT



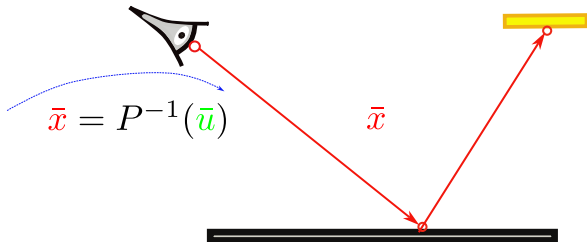
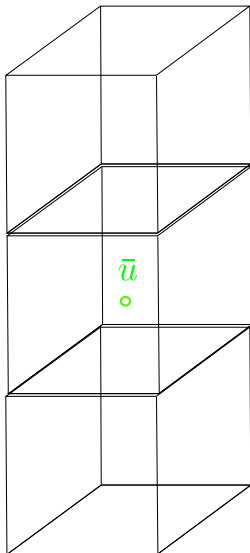


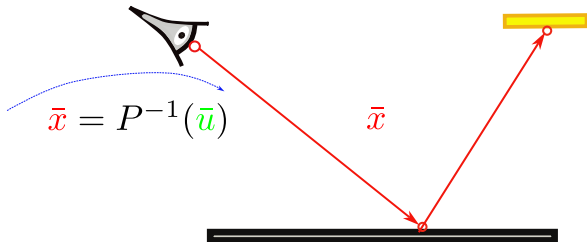
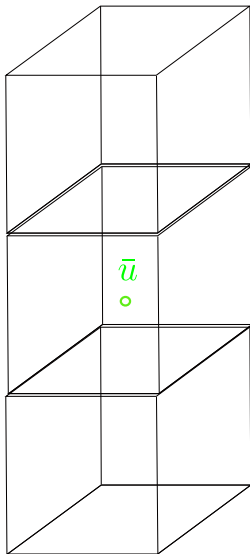
Uses MIS to combine multiple sampling techniques





Multiplexed MLT



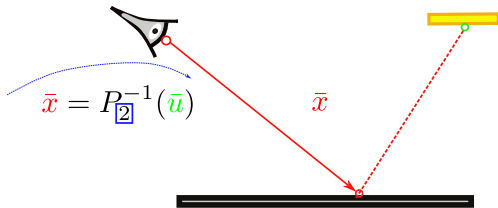
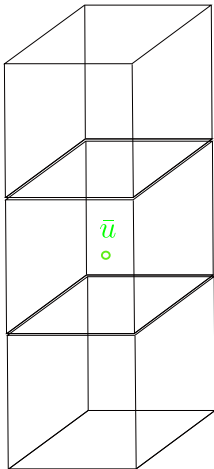




tech. 1

tech. 2

tech. 3



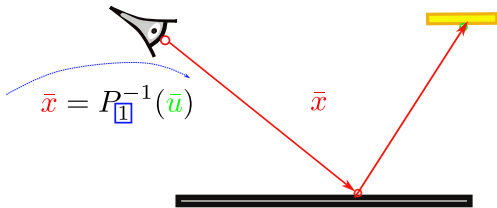
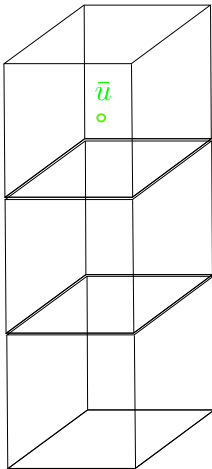
Sample the path using tech. 2



tech. 1

tech. 2

tech. 3



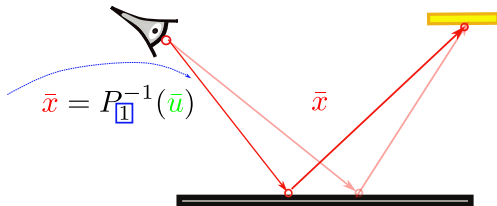
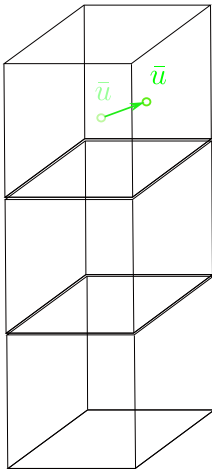
Sample the path using tech. 1



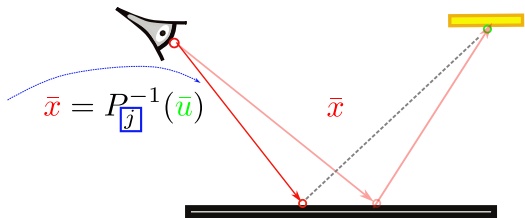
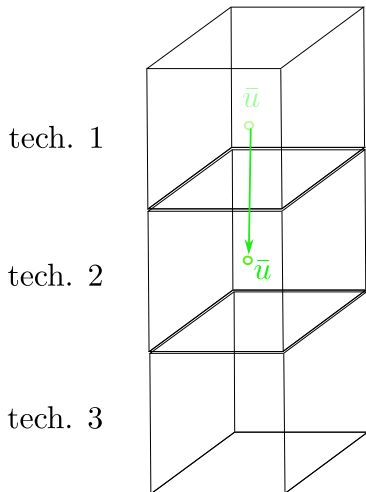
tech. 1

tech. 2

tech. 3



Sample the path using tech. 1



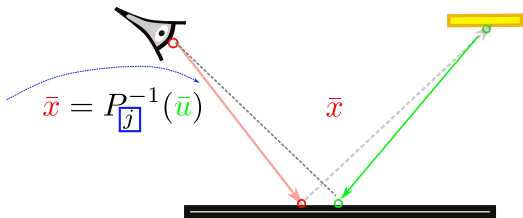
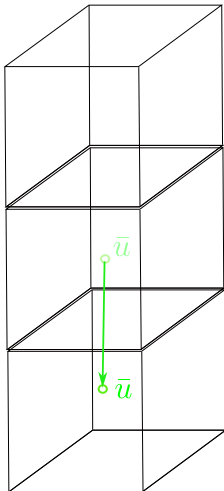
Inter-technique mutation



tech. 1

tech. 2

tech. 3



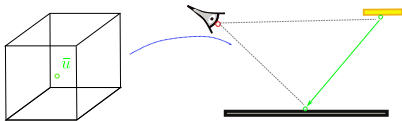
Inter-technique mutation



PSSMLT + BDPT $\neq$ Multiplexed MLT

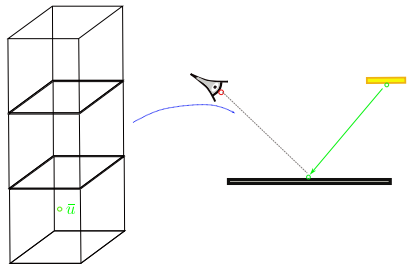


PSSMCMC + BDPT



One sample = Set of BDPT connections

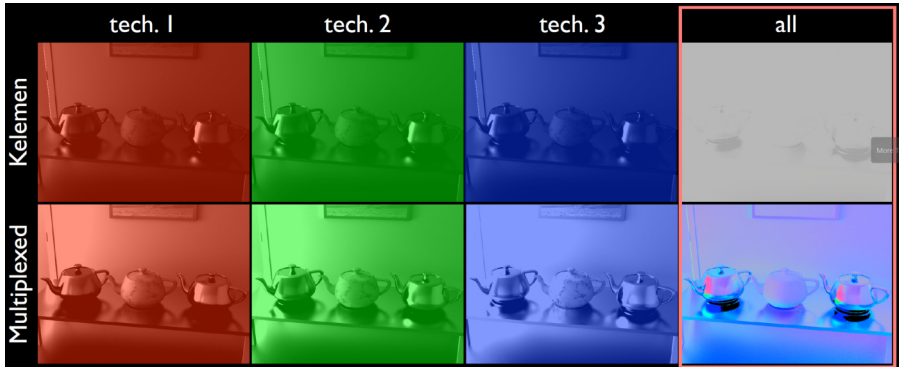
MMLT



One sample = One path



PSSMLT + BDPT $\neq$ Multiplexed MLT



of samples per technique.



of samples with MMLT

$\approx$



Actual relative contributions



Like in the case of BDPT, we define an (s, t) technique (or t -th technique given fixed k) as a bidirectional path sampling procedure where we trace t eye-subpath and s light-subpath vertices to generate a path of length $k = s + t - 1$.

Each $p_t(\bar{x})$ then corresponds to a PDF of sampling such a pair of subpaths and is defined as a product of subpath sampling probability densities with t eye subpath vertices and $s = (k + 1) - t$ light subpaths vertices for a given path length k .

Generalizing the original PSSMLT approximation

$$I_j \approx \frac{1}{N} \sum_{i=1}^N h_j(\bar{x}_i) \frac{f(\bar{x}_i)}{p(\bar{x}_i)} = \frac{1}{N} \sum_{i=1}^N h_j(\bar{x}_i) C_j(\bar{x}_i),$$

where $C_j(\bar{x}) = f(\bar{x})/p(\bar{x})$ is the path contribution, Multiple Importance Sampling (MIS) uses ordinary MC integration to estimate I_j by taking N_t samples $\bar{x}_{t,i}$ ($t = 1 \dots M, i = 1 \dots N_t$) distributed according to $p_t(\bar{x})$:

$$I_j \approx \sum_{t=1}^M \frac{1}{N_t} \sum_{i=1}^{N_t} h_j(\bar{x}_{t,i}) w_t(\bar{x}_{t,i}) C_t(\bar{x}_{t,i}). \quad (4)$$

This equations show that PSSMLT is an MCMC sampling approach operating on a special case of MIS, where we use only one technique $p(\bar{x})$ and $w(\bar{x}) = 1$.

Each target distribution $f^*(\bar{x}, T_t) = w_t(\bar{x})C_t^*(\bar{x})p_t(\bar{x})$ in the primary space is “warped” according to the corresponding CDF $P_t(\bar{x})$ leading to

$$\hat{f}^*(\bar{u}, T_t) = \hat{w}_t(\bar{u})\hat{C}_t^*(\bar{u}) \text{ for each technique } t,$$

since $w_t(\bar{x})C_t^*(\bar{x})p_t(\bar{x})d\mu(\bar{x}) = \hat{w}_t(\bar{u})\hat{C}_t^*(\bar{u})d\bar{u}$, where $\hat{w}_t(\bar{u})$ is defined analogously to $\hat{C}_t^*(\bar{u})$.

Each t -th primary space has its own mapping defined as $P_t^{-1}(\bar{u})$, thus we can utilize the additional knowledge of $p_t(\bar{x})$ and $P_t^{-1}(\bar{u})$. The proposed state $\bar{v}$, given the current state $\bar{u}$ and the technique t , is accepted with the probability

$$\min(1, a(\bar{v}|\bar{u})), \text{ where } a(\bar{v}|\bar{u}) = \frac{\hat{w}_t(\bar{v})\hat{C}_t^*(\bar{v})q(\bar{u}|\bar{v})}{\hat{w}_t(\bar{u})\hat{C}_t^*(\bar{u})q(\bar{v}|\bar{u})}.$$



- ▶ Like in PSSMLT, the state of the Markov chain is a vector of random numbers $\bar{u}_k \in [0; 1]^{O(k)}$ used for path sampling. The actual number of components in $\bar{u}_k$ depends on the implementation (e.g., if each vertex uses three random numbers then $\bar{u}_k \in [0; 1]^{3(k+1)}$).
- ▶ There is also the extra state $u_{k,t} \in [0; 1)$ to specify the sampling technique $P_t^{-1}(\bar{u})$. This extra state determines the number of vertices for the eye and light subpaths for a BPT connection.
- ▶ For paths of length k , the eye subpath consists of $t = b(k+2)u_{k,t} \in [0, k+1]$ vertices and the light subpath of $s = (k+1) - t$ vertices (there are $k+2$ possible techniques). The two subpaths are then connected deterministically to compute the path contribution. Unlike bidirectional path tracing, subpaths are connected only at their endpoints.

- ▶ The **key difference** to PSSMLT is that, because of the additional term $\hat{w}_t(\bar{u})$, we do not need $p_t(\bar{x})$ to fit $f^*(\bar{x})$ well for all $\bar{x}$, but only locally where $w_t(\bar{x})$ is large enough.
- ▶ Being a primary space approach, the method maintains the **ease of implementation**. This extra flexibility is naturally combined with the MIS framework.
- ▶ A provably optimal weighting function such as the balance heuristic ensures that $w_t(\bar{x})$ automatically gets small in the region where $p_t(\bar{x})$ is a bad approximation of $f^*(\bar{x})$.
- ▶ The probability of switching from the t -th to the t' -th target distribution then becomes:

$$r(t'|t) = \frac{\hat{w}_{t'}(\bar{u})\hat{C}_{t'}^*(\bar{u})q(t|t')}{\hat{w}_t(\bar{u})\hat{C}_t^*(\bar{u})q(t'|t)}.$$



Example: Original MLT



Equal-time comparisons of the methods



Equal-time comparisons of the methods



Equal-time comparisons of the methods



- + MCMC is more robust to complex lighting, i.e. modern MCMC method survive the curse of dimensionality better than ordinary MC methods (like PT and BDPT).
 - + Better survives the curse of dimensionality
 - + Useful for $\geq 7 - 10$ bounces, so, whenever the majority of light transport in the image is high-dimensional (that is, roughly more than 7-10 bounces), MCMC usually behaves better in such scenarios.
 - + Robust at highly occluded and glossy scenes, i.e. it helps exploring narrow islands and peaks in path space caused by highly occluded lighting configuration and/or highly glossy materials.
- MCMC is famous for its **non-uniform convergence**, that is, it might take a while before Markov chain finds some important feature on the image, like some distant reflected caustic. E.g., a feature can suddenly appear on the image rendered for a long time which is especially **bad for animation** and previews.
- Specular-diffuse-specular paths cannot be captured. Indeed, cinema industry mainly uses OMC-based methods, and instead of Markov chains they prefer to use path guiding.



This class of methods based on the selection of sample population. They are called **Population Monte Carlo (PMC)** methods Bashford-Rogers et al., 2021; Cline et al., 2005; Lai et al., 2007.

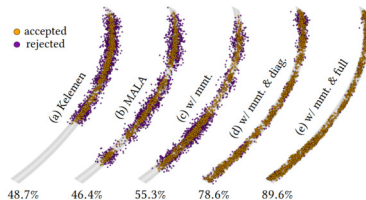
Idea: Keep information about the samples in time and then reuse the best samples as the starting state for small steps (mutations).

Advantages:

- ▶ Easier to parallelize due to short chains. This is extremely important for the implementation of the MCMC on GPUs
- ▶ It is more interactive than the conventional MCMC algorithms with long chains (i.e. the images look more realistic at the beginning of the rendering)
- ▶ PMC can compute the direct illumination, which is strongly not recommended in all kinds of the MLT.
 - Lower accuracy in the limit compared with the MLT Frolov et al., 2021



2D sampling example (from Luan et al., 2020): Comparison of various MCMC sampling algorithms on a simple two-dimensional anisotropic geometry, typical of those encountered in the primary sample space near caustics. Starting at the same initial position (top of the curve), we show 2048 samples and acceptance ratios produced by each algorithm. "Mmt." refers to momentum, and "diag." and "full" to diagonal and full preconditioning, respectively.



- ▶ Manifold exploration mutation (2012) Jakob and Marschner, 2012
- ▶ Reversible jump metropolis light transport using inverse mappings (2017) Bitterli et al., 2017
- ▶ Delayed rejection Metropolis light transport (2020) Rioux-Lavoie et al., 2020
- ▶ Langevin monte carlo rendering with gradient-based adaptation (2020) Luan et al., 2020



- [1] T. Bashford-Rogers, L. P. Santos, D. Marnerides, and K. Debattista. “Ensemble metropolis light transport”. In: *ACM Transactions on Graphics (TOG)* 41.1 (2021), pp. 1–15.
- [2] B. Bitterli, W. Jakob, J. Novák, and W. Jarosz. “Reversible jump metropolis light transport using inverse mappings”. In: *ACM Transactions on Graphics (TOG)* 37.1 (2017), pp. 1–12.
- [3] D. Cline, J. Talbot, and P. Egbert. “Energy redistribution path tracing”. In: *ACM Transactions on Graphics (TOG)* 24.3 (2005), pp. 1186–1195.
- [4] V. A. Frolov, A. G. Voloboy, S. Ershov, and V. A. Galaktionov. “Light transport in realistic rendering: state-of-the-art simulation methods”. In: *Programming and Computer Software* 47 (2021), pp. 298–326.
- [5] T. Hachisuka, A. S. Kaplanyan, and C. Dachsbacher. “Multiplexed metropolis light transport”. In: *ACM Transactions on Graphics (TOG)* 33.4 (2014), pp. 1–10.
- [6] W. Jakob and S. Marschner. “Manifold exploration: A markov chain monte carlo technique for rendering scenes with difficult specular transport”. In: *ACM Transactions on Graphics (TOG)* 31.4 (2012), pp. 1–13.



- [7] C. Kelemen, L. Szirmay-Kalos, G. Antal, and F. Csonka. “A simple and robust mutation strategy for the metropolis light transport algorithm”. In: *Computer Graphics Forum*. Vol. 21. 3. Wiley Online Library. 2002, pp. 531–540.
- [8] J. Krivanek, A. Keller, I. Georgiev, A. S. Kaplanyan, M. Fajardo, M. Meyer, J.-D. Nahmias, O. Karljk, and J. Canada. “Recent advances in light transport simulation: some theory and a lot of practice”. In: *ACM SIGGRAPH 2014 Courses* (2014), pp. 1–6.
- [9] Y.-C. Lai, S. H. Fan, S. Chenney, and C. Dyer. “Photorealistic image rendering with population monte carlo energy redistribution”. In: *Proceedings of the 18th Eurographics conference on Rendering Techniques*. 2007, pp. 287–295.
- [10] F. Luan, S. Zhao, K. Bala, and I. Gkioulekas. “Langevin monte carlo rendering with gradient-based adaptation.”. In: *ACM Trans. Graph.* 39.4 (2020), p. 140.
- [11] D. Rioux-Lavoie, J. Litalien, A. Gruson, T. Hachisuka, and D. Nowrouzezahrai. “Delayed rejection Metropolis light transport”. In: *ACM Transactions on Graphics (TOG)* 39.3 (2020), pp. 1–14.
- [12] M. Šik and J. Křivánek. “Survey of Markov chain Monte Carlo methods in light transport simulation”. In: *IEEE transactions on visualization and computer graphics* 26.4 (2018), pp. 1821–1840.



- [13] E. Veach and L. J. Guibas. “Metropolis light transport”. In: *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*. 1997, pp. 65–76.